

MI JPD API

Version 2.0

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
2.0	Initial release	12/17/2020
	Added PROCFS introduction	08/25/2021

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. 概述.....	1
1.1. 模块简介.....	1
1.2. 解码流程图.....	1
1.2.1 Tiramisu 解码流程图.....	1
1.2.2 Muffin 解码流程图.....	2
1.2.3 Mochi 解码流程图.....	3
1.3. 关键词说明.....	3
1.3.1 JPG/JPEG.....	3
1.3.2 JPD.....	3
1.3.3 STR.....	3
1.3.4 APP.....	3
2. API 参考.....	4
2.1. MI_JPD_CreateDev.....	4
2.2. MI_JPD_DestroyDev.....	5
2.3. MI_JPD_CreateChn.....	6
2.4. MI_JPD_DestroyChn.....	8
2.5. MI_JPD_GetChnAttr.....	9
2.6. MI_JPD_StartChn.....	10
2.7. MI_JPD_StopChn.....	11
2.8. MI_JPD_GetChnStatus.....	11
2.9. MI_JPD_ResetChn.....	12
2.10. MI_JPD_GetStreamBuf.....	13
2.11. MI_JPD_PutStreamBuf.....	15
2.12. MI_JPD_DropStreamBuf.....	16
2.13. MI_JPD_QueryStreamInfo.....	17
2.14. MI_JPD_DirectBufDecode.....	18
3. 数据类型.....	21
3.1. MI_JPD_DEV.....	21
3.2. MI_JPD_CHN.....	21
3.3. MI_JPD_ErrCode_e.....	22
3.4. MI_JPD_InitParam_t.....	22
3.5. MI_JPD_ChnCreatConf_t.....	23
3.6. MI_JPD_ChnAttr_t.....	23
3.7. MI_JPD_ChnStatus_t.....	24
3.8. MI_JPD_StreamBuf_t.....	25
3.9. MI_JPD_StreamInfo_t.....	27
3.10. MI_JPD_DirectInputBuf_t.....	28
3.11. MI_JPD_DirectOutputBuf_t.....	28
4. 错误码.....	30
5. PROCFS 介绍.....	31
5.1. cat.....	31

5.2. echo.....	33
----------------	----

1. 概述

1.1. 模块简介

JPD 解码模块提供创建解码通道、控制解码及传送码流等功能。

JPD 模块的输入源主要是两类：

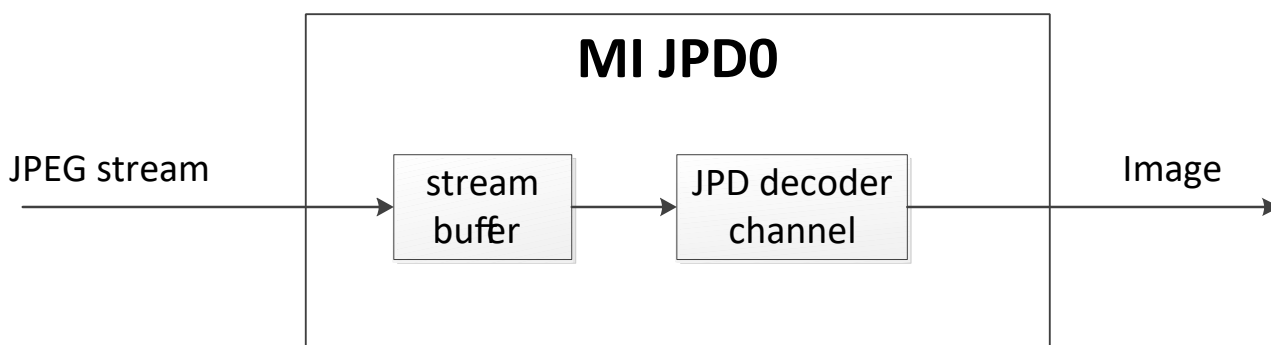
- APP 读取码流文件向解码模块发送数据；
- APP 将网络接收到的码流数据直接发送到解码模块。

表 1-1芯片解码规格

芯片	支持最大通道数	支持协议	分辨率范围	最大性能
Tiramisu	16	JPEG	max 8192x8192, min 16x8	3840x2160@20fps
Muffin	16*4	JPEG	max 8192x8192, min 16x8	3840x2160@20fps
Mochi	16*2	JPEG	max 8192x8192, min 16x8	3840x2160@20fps

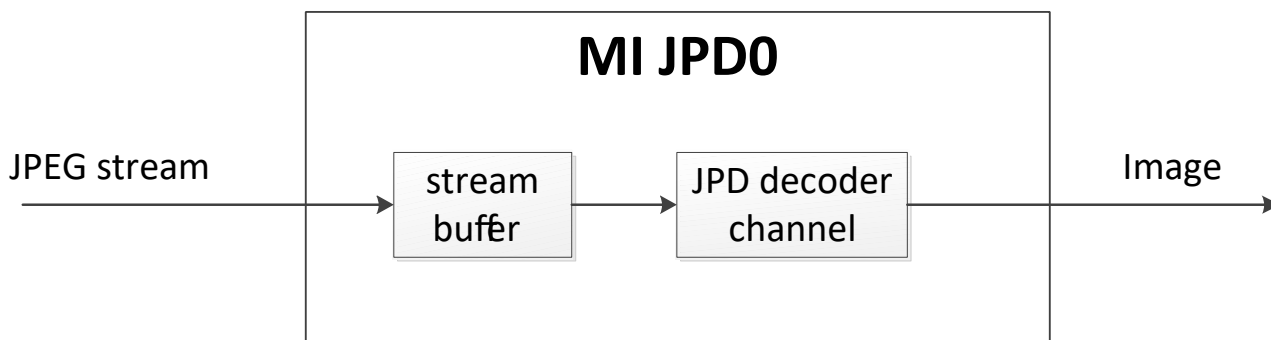
1.2. 解码流程图

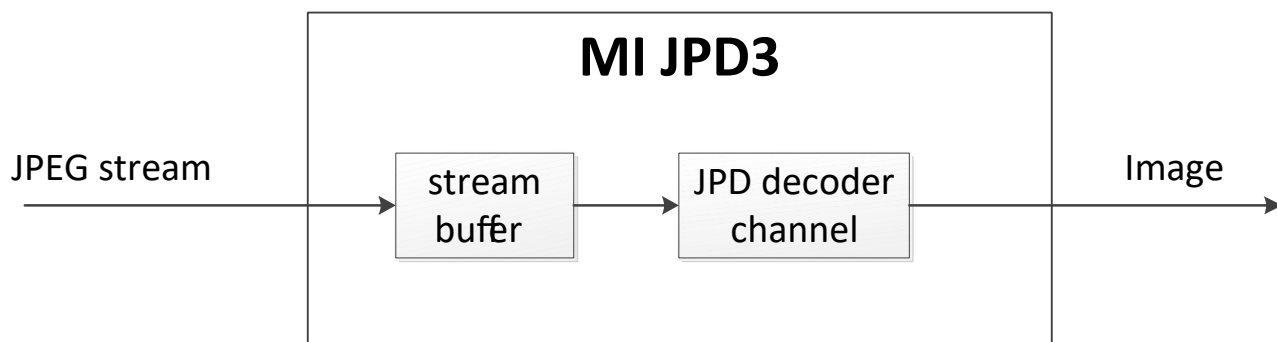
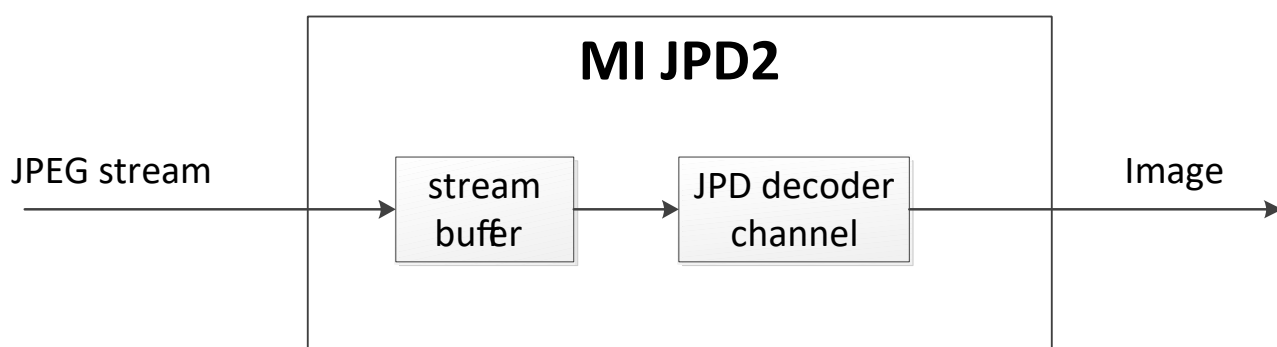
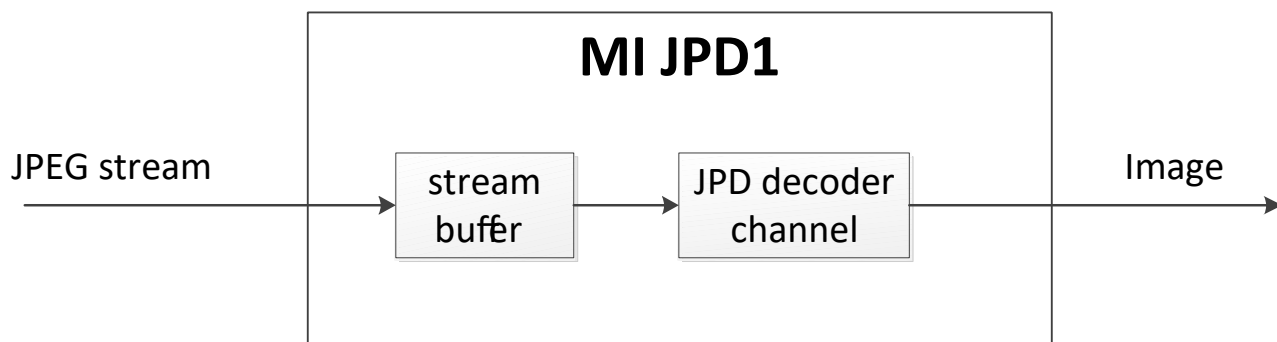
1.2.1 Tiramisu 解码流程图



Tiramisu 平台只支持 1 个 device，device id 为 0。

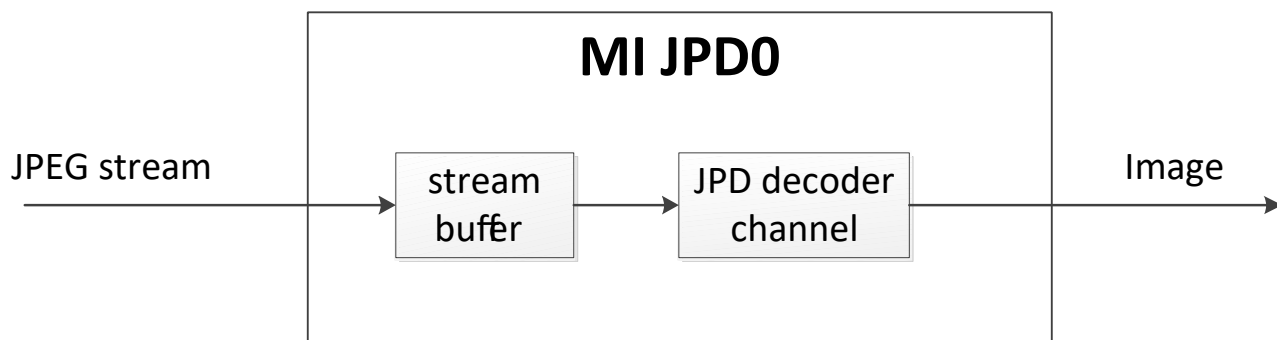
1.2.2 Muffin 解码流程图

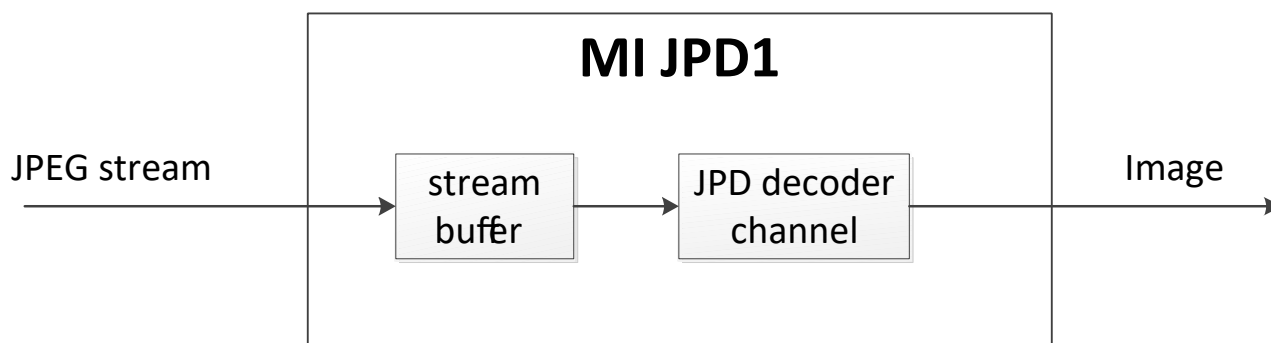




Muffin 平台支持 4 个 device，device id 分别为 0、1、2、3。

1.2.3 Mochi 解码流程图





Mochi 平台支持 2 个 device，device id 分别为 0、1。

1.3. 关键词说明

1.3.1 JPG/JPEG

Joint Photographic Experts Group: 联合摄影专家组，该标准由该组织制定。

1.3.2 JPD

JPG Decoder: JPG 解码器

1.3.3 STR

Suspend To RAM: 指系统关机或进入省电模式后，除内存外其他硬件断电，达到兼容低功耗的同时快速重启机的效果。

1.3.4 APP

Application: 应用。

2. API 参考

该功能模块提供以下 API:

API名	功能
MI_JPD_CreateDev	创建设备。
MI_JPD_DestroyDev	销毁设备。
MI_JPD_CreateChn	创建解码通道。
MI_JPD_DestroyChn	销毁解码通道。
MI_JPD_GetChnAttr	获取解码通道属性。
MI_JPD_StartChn	解码通道开始接收码流。
MI_JPD_StopChn	解码通道停止接收码流。
MI_JPD_GetChnStatus	获取解码通道状态。
MI_JPD_ResetChn	重置解码通道。
MI_JPD_GetStreamBuf	从解码通道获取指定长度的码流缓存。
MI_JPD_PutStreamBuf	通知解码通道送流成功，解码通道开始解码。
MI_JPD_DropStreamBuf	通知解码通道送流失败，解码通道会释放对应通过 MI_JPD_GetStreamBuf 所取到的码流缓存，与 MI_JPD_PutStreamBuf 互斥调用。
MI_JPD_QueryStreamInfo	从解码通道获取码流信息。
MI_JPD_DirectBufDecode	解码通道直接按APP指定的输入和输出缓存解码。

2.1. MI_JPD_CreateDev

➤ 功能

创建设备。

➤ 定义

MI_S32 MI_JPD_CreateDev(MI_JPD_DEV JpdDev, MI_JPD_InitParam_t *pstInitParam);

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
pstInitParam	初始化参数，目前未起实际作用，保留为方便未来扩展。 请参考 MI_JPD_InitParam_t	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INITED	设备已经初始化。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- 此函数可以不调用，但 [MI_JPD_CreateChn](#) 需要指定有效的 device id，内部会创建相应的 device。
- 本接口不可重复调用，需要和 [MI_JPD_DestroyDev](#) 成对使用，否则返回失败。
- Muffin 芯片设置 MI_SYS_BindType_e 参数时有如下限制：
- 只有 device 0 与 device 1 支持使用设置 MI_SYS_BindType_e 为 E_MI_SYS_BIND_TYPE_REALTIME 类型，device 2 与 device 3 不支持设置 MI_SYS_BindType_e 为 E_MI_SYS_BIND_TYPE_REALTIME 类型。
- 同一时刻只支持 1 个 device 可以设置 MI_SYS_BindType_e 为 E_MI_SYS_BIND_TYPE_REALTIME 类型，不能有多个 device 同时设置为 E_MI_SYS_BIND_TYPE_REALTIME 类型。例如：如果 device 0 成功设置为 E_MI_SYS_BIND_TYPE_REALTIME 类型，此时 device 1 设置 E_MI_SYS_BIND_TYPE_REALTIME 类型会失败。

➤ 举例

无。

➤ 相关主题

[MI_JPD_DestroyDev](#)

2.2. MI_JPD_DestroyDev

➤ 功能

销毁设备。

➤ 定义

```
MI_S32 MI_JPD_DestroyDev(MI_JPD_DEV JpdDev);
```

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_NOT_INIT	设备未初始化。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- 此函数必须在创建设备后调用，否则返回失败。
- 如果本接口在app退出前没有调用，内部会自动销毁设备。
- 本接口不可重复调用，需要和 [MI_JPD_CreateDev](#) 成对使用，否则返回失败。

➤ 举例

无。

➤ 相关主题

[MI_JPD_CreateDev](#)

2.3. MI_JPD_CreateChn

➤ 功能

创建解码通道。

➤ 定义

MI_S32 MI_JPD_CreateChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_JPD_ChnCreatConf_t *pstChnCreatConf)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
pstChnCreatConf	解码通道属性指针，请参考 MI_JPD_ChnCreatConf_t	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_NOT_INIT	设备未初始化。
MI_ERR_JPD_ILLEGAL_PARAM	非法参数，如输入参数超过通道解码能力。
MI_ERR_JPD_CHN_EXIST	试图创建已经存在的通道。
MI_ERR_JPD_NOMEM	分配内存失败（如系统内存不足）。
MI_ERR_JPD_NULL_PTR	输入参数为空指针。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

➤ 芯片差异

芯片名	最大解码通道号
Tiramisu	16
Muffin	16

※ 注意

- 通道号不能超出最大的通道号范围。

➤ 举例

```
MI_S32 StartJpd(void)
{
    MI_S32 s32Ret = MI_ERR_JPD_FAILED;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;
    MI_JPD_ChnCreatConf_t stChnCreatConf;

    memset(&stChnCreatConf, 0x0, sizeof(MI_JPD_ChnCreatConf_t));
    stChnCreatConf.u32StreamBufSize = 0x100000;
    stChnCreatConf.u32MaxPicWidth = 1920;
    stChnCreatConf.u32MaxPicHeight = 1080;
    s32Ret = MI_JPD_CreateChn(JpdDev, JpdChn, & stChnCreatConf);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_JPD_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    s32Ret = MI_JPD_StartChn(JpdDev, JpdChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_JPD_StartChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    return s32Ret;
}
```

➤ 相关主题

[MI_JPD_DestroyChn](#)

2.4. MI_JPD_DestroyChn

➤ 功能

销毁解码通道。

➤ 定义

MI_S32 MI_JPD_DestroyChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_NOT_STOP	停止接收码流前不允许关闭通道。
MI_ERR_JPD_CHN_UNEXIST	通道不存在。
MI_ERR_JPD_NOT_INIT	设备未初始化。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- 销毁前需停止接收码流。

➤ 举例

```
MI_S32 StopJpd(void)
{
    MI_S32 s32Ret = MI_ERR_JPD_FAILED;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;

    // Destroy send stream thread

    s32Ret = MI_JPD_StopChn(JpdDev, JpdChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_JPD_StopChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
}
```

```
s32Ret = MI_JPD_DestroyChn(JpdDev, JpdChn);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_JPD_DestroyChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

return s32Ret;
}
```

➤ 相关主题

[MI_JPD_CreateChn](#)

2.5. MI_JPD_GetChnAttr

➤ 功能

获取解码通道属性。

➤ 定义

MI_S32 MI_JPD_GetChnAttr(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_JPD_ChnAttr_t *pstChnAttr)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
pstChnAttr	解码通道属性指针。请参考 MI_JPD_ChnAttr_t	输出

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_ILLEGAL_PARAM	非法参数。
MI_ERR_JPD_CHN_UNEXIST	通道不存在。
MI_ERR_JPD_NOT_INIT	设备未初始化。
MI_ERR_JPD_NULL_PTR	输入参数为空指针。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

无。

- 举例
无。
- 相关主题
无。

2.6. MI_JPD_StartChn

- 功能
通道开始接收码流。
- 定义
MI_S32 MI_JPD_StartChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

- 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入

- 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_JPD_NOT_INIT	设备未初始化。

- 依赖
 - 头文件：mi_jpd.h
 - 库文件：libmi_jpd.a/libmi_jpd.so

- ※ 注意

- 启动接收码流前必须保证通道已创建，否则会返回通道未创建的错误码 MI_ERR_JPD_CHN_UNEXIST。
- 启动接收码流之后，才能调用 MI_JPD_GetStreamBuf/MI_JPD_PutStreamBuf 发送码流。
- 重复调用启动接收码流接口时，返回成功。

- 举例
请参见[MI_JPD_CreateChn](#)的举例。

- 相关主题
[MI_JPD_StopChn](#)

2.7. MI_JPD_StopChn

- 功能
通道停止接收码流。

➤ 定义

MI_S32 MI_JPD_StopChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_NOT_START	通道未使能。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_JPD_NOT_INIT	设备未初始化。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- 调用此接口后，调用发送码流接口 MI_JPD_GetStreamBuf/MI_JPD_PutStreamBuf 会返回失败。
- 重复调用停止接收码流接口时，返回MI_ERR_JPD_CHN_NOT_START。

➤ 举例

请参见[MI_JPD_DestroyChn](#)的举例。

➤ 相关主题

[MI_JPD_StartChn](#)

2.8. MI_JPD_GetChnStatus

➤ 功能

查询解码通道状态。

➤ 定义

MI_S32 MI_JPD_GetChnStatus(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_JPD_ChnStatus_t *pstChnStatus)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
pstChnStatus	解码通道状态结构体指针。请参考 MI_JPD_ChnStatus_t	输出

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或已销毁。
MI_ERR_JPD_ILLEGAL_PARAM	非法参数。
MI_ERR_JPD_NULL_PTR	输入参数为空指针。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

2.9. MI_JPD_ResetChn

➤ 功能

复位解码通道。

➤ 定义

MI_S32 MI_JPD_ResetChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_NOT_START	通道未使能。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或已销毁。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意
无。

➤ 举例
无。

➤ 相关主题
无。

2.10. MI_JPD_GetStreamBuf

➤ 功能
从解码通道获取指定长度的码流缓存。

➤ 定义
MI_S32 MI_JPD_GetStreamBuf(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_U32 u32RequiredLength, MI_JPD_StreamBuf_t *pstRetStreamBuf, MI_S32 s32MilliSecToWait)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
u32RequiredLength	期望申请到的码流缓存长度。	输入
pstRetStreamBuf	底层返回分配成功的码流缓存指针。请参考 MI_JPD_StreamBuf_t	输出
s32MilliSecToWait	设定请求超时时间参数。取值范围： -1：阻塞。 0：非阻塞。 正值：超时时间，单位为毫秒。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_ILLEGAL_PARAM	非法参数。
MI_ERR_JPD_CHN_NOT_START	通道未使能。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或者已销毁。
MI_ERR_JPD_BUF_FULL	底层的码流缓冲区已满。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- 调用该接口成功后，将对应JPG码流拷贝到该接口返回的地址空间，如果成功，最后需要调用[MI_JPD_PutStreamBuf](#)；如果失败，最后需要调用[MI_JPD_DropStreamBuf](#)。

- 目前只支持按Frame送流方式，即每次送完整的一帧JPG码流数据。

➤ 举例

```
MI_S32 JpdSendStreamThreadFunc(void)
{
    MI_S32 s32Ret = MI_ERR_JPD_FAILED;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;
    MI_U32 u32RequiredLength = 0;
    MI_S32 s32MilliSec = 0;
    MI_JPD_StreamBuf_t stRetStreamBuf;

    do {
        //Check if you need stop sending stream
        if (g_bStop)
        {
            break;
        }

        //get one stream from file or ipc, then assign u32RequiredLength to the size of the
        current stream

        memset(&stRetStreamBuf, 0x0, sizeof(MI_JPD_StreamBuf_t));
        s32MilliSec = 30; //timeout is 30ms
        s32Ret = MI_JPD_GetStreamBuf(JpdDev, JpdChn, u32RequiredLength, &stRetStreamBuf,
            s32MilliSec);
        if (MI_SUCCESS != s32Ret)
        {
            printf("MI_JPD_GetStreamBuf failed, s32Ret: 0x%x.\n", s32Ret);
            continue;
        }

        if (stRetStreamBuf.u32HeadLength+stRetStreamBuf.u32TailLength <
            u32RequiredLength)
        {
            //something wrong happen
            printf("MI_JPD_GetStreamBuf return wrong value: HeadLen%u TailLen%u
                RequiredLength%u\n",
                stRetStreamBuf.u32HeadLength, stRetStreamBuf.u32TailLength,
                u32RequiredLength);
            s32Ret = MI_JPD_DropStreamBuf(JpdDev, JpdChn, &stRetStreamBuf);
            if (MI_SUCCESS != s32Ret)
            {
                printf("MI_JPD_DropStreamBuf failed, s32Ret: 0x%x.\n", s32Ret);
            }
        }
    }
}
```

```

        continue;
    }
}
else
{
    memcpy(stRetStreamBuf.pu8HeadVirtAddr, addr_of_stream,
           min(stRetStreamBuf.u32HeadLength, u32RequiredLength));
    if (stRetStreamBuf.u32TailLength > 0)
        memcpy(stRetStreamBuf.pu8TailVirtAddr,
               addr_of_stream+stRetStreamBuf.u32HeadLength,
               min(stRetStreamBuf.u32TailLength, u32RequiredLength-
                 stRetStreamBuf.u32HeadLength));
}

stRetStreamBuf.u32ContentLength = u32RequiredLength;
s32Ret = MI_JPD_PutStreamBuf(JpdDev, JpdChn, &stRetStreamBuf);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_JPD_PutStreamBuf failed, s32Ret: 0x%x.\n", s32Ret);
    continue;
}
}while(!g_bStop);

return MI_SUCCESS;
}

```

➤ 相关主题

[MI_JPD_PutStreamBuf](#)
[MI_JPD_DropStreamBuf](#)

2.11. MI_JPD_PutStreamBuf

➤ 功能

通知解码通道送流成功，解码通道开始解码。

➤ 定义

MI_S32 MI_JPD_PutStreamBuf(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_JPD_StreamBuf_t *pstRetStreamBuf)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
pstRetStreamBuf	先前 MI_JPD_GetStreamBuf 获取到的缓存指针。请参考 MI_JPD_StreamBuf_t 。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_NOT_START	通道未使能。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或已销毁。

➤ 依赖

- 头文件: mi_jpd.h
- 库文件: libmi_jpd.a/libmi_jpd.so

※ 注意

- 调用MI_JPD_GetStreamBuf成功后, 将对应JPG码流拷贝到该接口返回的地址空间, 如果成功, 最后需要调用MI_JPD_PutStreamBuf; 如果失败, 最后需要调用MI_JPD_DropStreamBuf。

➤ 举例

请参见 [MI_JPD_GetStreamBuf](#) 的举例。

➤ 相关主题

[MI_JPD_GetStreamBuf](#)

2.12. MI_JPD_DropStreamBuf

➤ 功能

通知解码通道送流失败, 解码通道会释放对应通过[MI_JPD_GetStreamBuf](#)所取到的码流缓存。

➤ 定义

MI_S32 MI_JPD_DropStreamBuf(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_JPD_StreamBuf_t *pstRetStreamBuf)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
pstRetStreamBuf	先前MI_JPD_GetStreamBuf获取到的缓存指针。请参考 MI_JPD_StreamBuf_t	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_NOT_START	通道未使能。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或已销毁。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- 调用MI_JPD_GetStreamBuf成功后，将对应JPG码流拷贝到该接口返回的地址空间，如果成功，最后需要调用MI_JPD_PutStreamBuf；如果失败，最后需要调用MI_JPD_DropStreamBuf。

➤ 举例

请参见 [MI_JPD_GetStreamBuf](#) 的举例。

➤ 相关主题

[MI_JPD_GetStreamBuf](#)

2.13. MI_JPD_QueryStreamInfo

➤ 功能

从解码通道获取码流信息。

➤ 定义

MI_S32 MI_JPD_QueryStreamInfo(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_JPD_DirectInputBuf_t *pstInputBuf, MI_JPD_StreamInfo_t *pstStreamInfo)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
pstInputBuf	存放输入缓存的指针。请参考 MI_JPD_DirectInputBuf_t 。	输入
pstStreamInfo	存放码流信息的指针。请参考 MI_JPD_StreamInfo_t 。	输出

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_NOT_START	通道未使能。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或已销毁。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- u32BufWidth/u32BufHeight指的是输出缓存的实际宽/高，而u32CurPicWidth/u32CurPicHeight则是指图像的有效宽/高。

➤ 举例

请参见 [MI_JPD_DirectBufDecode](#) 的举例。

➤ 相关主题

[MI_JPD_DirectBufDecode](#)

2.14. MI_JPD_DirectBufDecode

➤ 功能

解码通道直接按APP指定的输入和输出缓存解码。

➤ 定义

MI_S32 MI_JPD_DirectBufDecode(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn,
MI_JPD_DirectInputBuf_t *pstInputBuf, MI_JPD_DirectOutputBuf_t *pstOutputBuf)

➤ 形参

参数名称	描述	输入/输出
JpdDev	设备号。	输入
JpdChn	解码通道号。	输入
pstInputBuf	存放输入缓存的指针。请参考 MI_JPD_DirectInputBuf_t 。	输入
pstOutputBuf	存放输出缓存的指针。请参考 MI_JPD_DirectOutputBuf_t 。	输入

➤ 返回值

返回值	描述
MI_SUCCESS	成功。
MI_ERR_JPD_FAILED	失败。
MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围。
MI_ERR_JPD_CHN_NOT_START	通道未使能。
MI_ERR_JPD_CHN_UNEXIST	通道未创建或已销毁。

➤ 依赖

- 头文件：mi_jpd.h
- 库文件：libmi_jpd.a/libmi_jpd.so

※ 注意

- 如果不清楚输出缓存的大小，APP在调用该接口之前需要先调用[MI_JPD_QueryStreamInfo](#)，输出缓存的大小是通过[MI_JPD_StreamInfo_t](#)中的u32BufWidth/u32BufHeight计算得出，而u32CurPicWidth/u32CurPicHeight则是指图像的实际有效宽高。

➤ 举例

```
MI_S32 directBufDecode(void)
{
    MI_S32 s32Ret = MI_SUCCESS;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;
```

```
MI_U8 *pu8Buf = NULL;
MI_U32 u32LengthCurStream = 0;
MI_JPD_DirectInputBuf_t stInputBuf;
MI_JPD_DirectOutputBuf_t stOutputBuf;
MI_JPD_StreamInfo_t stStreamInfo;
MI_U32 u32OutputBufSize = 0;

//get one jpg stream from file or ipc, assign its length to u32LengthCurStream
pu8Buf = malloc(u32LengthCurStream);
//copy one jpg stream from file or ipc to pu8Buf

memset(&stInputBuf, 0, sizeof(MI_JPD_DirectInputBuf_t));
memset(&stOutputBuf, 0, sizeof(MI_JPD_DirectOutputBuf_t));
memset(&stStreamInfo, 0, sizeof(MI_JPD_StreamInfo_t));

stInputBuf.pu8InputJPGRawFrameDataVirtAddr = pu8Buf;
stInputBuf.phyAddr = 0;
stInputBuf.u32Length = u32LengthCurStream;
s32Ret = MI_JPD_QueryStreamInfo(JpdDev, JpdChn, &stInputBuf, &stStreamInfo);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_JPD_QueryStreamInfo failed, s32Ret: 0x%x.\n", s32Ret);
    goto RETURN_FREE;
}
printf("stream info: w:%d h:%d format:%d bufW:%d bufH:%d\n",
        stStreamInfo.u32CurPicWidth, stStreamInfo.u32CurPicHeight,
        stStreamInfo.ePixelFormat, stStreamInfo.u32BufWidth,
        stStreamInfo.u32BufHeight);

// only support output pixel format YUYV422
stOutputBuf.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
stOutputBuf.u32Width = stStreamInfo.u32BufWidth;
stOutputBuf.u32Height = stStreamInfo.u32BufHeight;
stOutputBuf.u32Stride[0] = stOutputBuf.u32Width * 2;
u32OutputBufSize = stOutputBuf.u32Stride[0] * stOutputBuf.u32Height;
s32Ret = MI_SYS_MMA_Alloc(NULL, u32OutputBufSize, &stOutputBuf.phyAddr[0]);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_SYS_MMA_Alloc failed, s32Ret: 0x%x.\n", s32Ret);
    goto RETURN_FREE;
}

s32Ret = MI_JPD_DirectBufDecode(JpdDev, JpdChn, &stInputBuf, &stOutputBuf);
if (MI_SUCCESS != s32Ret)
```



```
{  
    printf("MI_JPD_DirectBufDecode failed, s32Ret: 0x%x.\n", s32Ret);  
    goto RETURN_FREE;  
}  
  
RETURN_FREE:  
    free(pu8Buf);  
    return s32Ret;  
}
```

➤ 相关主题

[MI_JPD_QueryStreamInfo](#)

3. 数据类型

解码相关数据类型、数据结构定义如下：

数据结构	说明
MI_JPD_DEV	定义JPD设备ID类型。
MI_JPD_CHN	定义JPD通道ID类型。
MI_JPD_ErrCode_e	定义码流错误码枚举体。
MI_JPD_InitParam_t	定义设备初始化参数结构体
MI_JPD_ChnCreatConf_t	定义通道初始化参数结构体。
MI_JPD_ChnAttr_t	定义通道属性结构体。
MI_JPD_ChnStatus_t	定义通道状态结构体。
MI_JPD_StreamBuf_t	定义码流缓存参数结构体。
MI_JPD_StreamInfo_t	定义码流信息结构体。
MI_JPD_DirectInputBuf_t	定义输入码流缓存参数结构体。
MI_JPD_DirectOutputBuf_t	定义输出图像缓存参数结构体。

3.1. MI_JPD_DEV

- 说明
定义 JPD 设备 ID 类型。
- 定义
#define MI_JPD_DEV MI_U32
- ※ 注意事项
无。
- 相关数据类型及接口
无。

3.2. MI_JPD_CHN

- 说明
定义 JPD 通道 ID 类型。

➤ 定义

```
#define MI_JPD_CHN MI_U32
```

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.3. MI_JPD_ErrCode_e

➤ 说明

定义码流错误码枚举体。

➤ 定义

```
typedef enum
{
    E_MI_JPD_ERR_CODE_UNKNOW = 0x0,
    E_MI_JPD_ERR_CODE_ILLEGAL_ACCESS,
    E_MI_JPD_ERR_CODE_DEC_TIMEOUT,
    E_MI_JPD_ERR_CODE_OUT_OF_MEMORY,
    E_MI_JPD_ERR_CODE_MAX
} MI_JPD_ErrCode_e;
```

➤ 成员

成员名称	描述
E_MI_JPD_ERR_CODE_UNKNOW	未知错误。
E_MI_JPD_ERR_CODE_ILLEGAL_ACCESS	不可访问，例如:未初始化或硬件出问题。
E_MI_JPD_ERR_CODE_DEC_TIMEOUT	解帧超时
E_MI_JPD_ERR_CODE_OUT_OF_MEMORY	内存不足

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.4. MI_JPD_InitParam_t

➤ 说明

定义设备初始化参数结构体。

➤ 定义

```
typedef struct MI_JPD_InitParam_s
{
    MI_U32 u32Reserved;
} MI_JPD_InitParam_t;
```

➤ 成员

成员名称	描述
u32Reserved	保留。

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_JPD_InitDev](#)

3.5. MI_JPD_ChnCreatConf_t

➤ 说明
定义通道初始化参数结构体。

➤ 定义

```
typedef struct MI_JPD_ChnCreatConf_s
{
    MI_U32 u32StreamBufSize;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U32 u32MaxPicWidth;
    MI_U32 u32MaxPicHeight;
} MI_JPD_ChnCreatConf_t;
```

➤ 成员

成员名称	描述
u32StreamBufSize	码流缓存总长度。
ePixelFormat	图像输入像素存储格式，目前支持semi plannar YUV420(即NV12)/YUYV422。参考MI_SYS_PixelFormat_e说明。
u32MaxPicWidth	解码图像最大宽。
u32MaxPicHeight	解码图像最大高。

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_JPD_CreateChn](#)

3.6. MI_JPD_ChnAttr_t

➤ 说明
定义通道属性结构体。

➤ 定义

```
typedef struct MI_JPD_ChnAttr_s
{
    MI_U32 u32StreamBufSize;
    MI_JPD_StreamInfo_t stStreamInfo;
```

} MI_JPD_ChnAttr_t;

➤ 成员

成员名称	描述
u32StreamBufSize	码流缓存总长度。
stStreamInfo	码流信息，参考 MI_JPD_StreamInfo_t 说明。

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_JPD_GetChnAttr](#)

3.7. MI_JPD_ChnStatus_t

➤ 说明

定义通道状态结构体。

➤ 定义

```
typedef struct MI_JPD_ChnStatus_s
{
    MI_U32 u32LeftStreamBytes;
    MI_U32 u32LeftStreamFrames;
    MI_U32 u32LeftPics;
    MI_U32 u32RecvStreamFrames;
    MI_U32 u32DecodeStreamFrames;
    MI_BOOL bChnStart;
    MI_JPD_ErrCode_e eErrCode;
} MI_JPD_ChnStatus_t;
```

➤ 成员

成员名称	描述
u32LeftStreamBytes	码流缓存中待解码的byte数。
u32LeftStreamFrames	码流缓存中待解码的帧数。
u32LeftPics	图像缓存中剩余的pic数目。
u32RecvStreamFrames	码流缓存中已接收码流帧数。
u32DecodeStreamFrames	码流缓存中已解码帧数。
bChnStart	解码器是否已经启动接收码流。
eErrCode	解码错误码信息。

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_JPD_GetChnStatus](#)

3.8. MI_JPD_StreamBuf_t

➤ 说明

定义码流缓存参数结构体。

➤ 定义

```
typedef struct MI_JPD_StreamBuf_s
{
    MI_U8 *pu8HeadVirtAddr;//ring buffer first part user mode VA
    MI_PHY u64HeadPhyAddr;//ring buffer first part PA
    MI_U32 u32HeadLength;//ring buffer first part Length
    MI_U8 *pu8TailVirtAddr;//ring buffer 2nd part user mode VA
    MI_PHY u64TailPhyAddr;//ring buffer 2nd part PA
    MI_U32 u32TailLength;//ring buffer 2nd part length
    MI_U32 u32ContentLength;//current used content length
}MI_JPD_StreamBuf_t;
```

➤ 成员

成员名称	描述
pu8HeadVirtAddr	头部码流缓存虚拟地址。
u64HeadPhyAddr	头部码流缓存物理地址。
u32HeadLength	头部码流缓存长度。
pu8TailVirtAddr	尾部码流缓存虚拟地址。
u64TailPhyAddr	尾部码流缓存物理地址。
u32TailLength	尾部码流缓存长度。
u32ContentLength	码流实际有效数据长度。

※ 注意事项

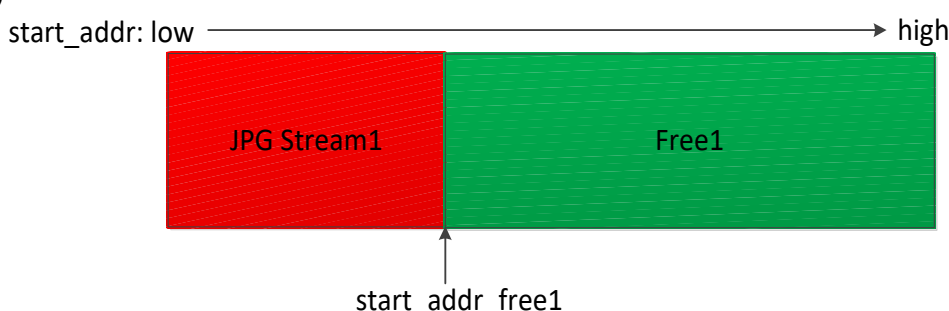
- 如果u32TailLength为0, 说明底层只返回了一块缓存pu8HeadVirtAddr, 其长度不小于上层期望得到的码流缓存长度, 上层需要将该帧码流拷贝到pu8HeadVirtAddr。此时底层码流缓存示意图可能有如下几种情况:

a)

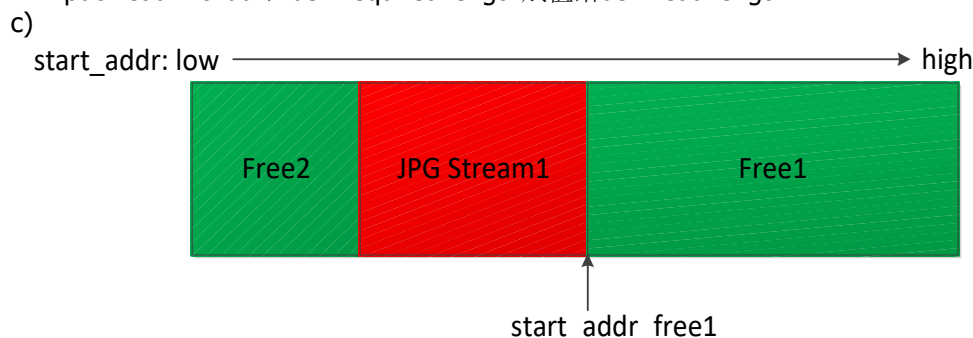


此时底层stream buffer都是空闲的, 如果Free1的长度满足于期望长度u32RequiredLength,底层会将 stream buffer 的 start_addr 赋值 给 pu8HeadVirtAddr , u32RequiredLength 赋值 给 u32HeadLength。

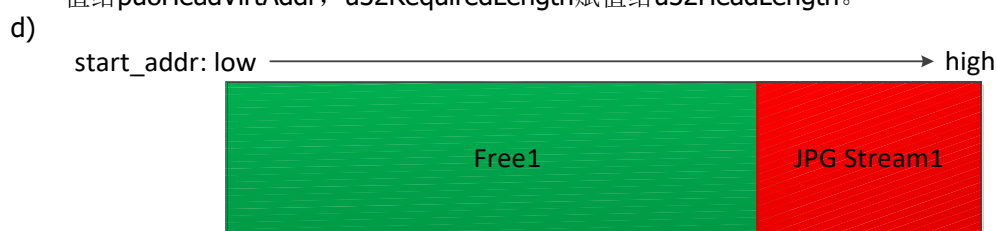
b)



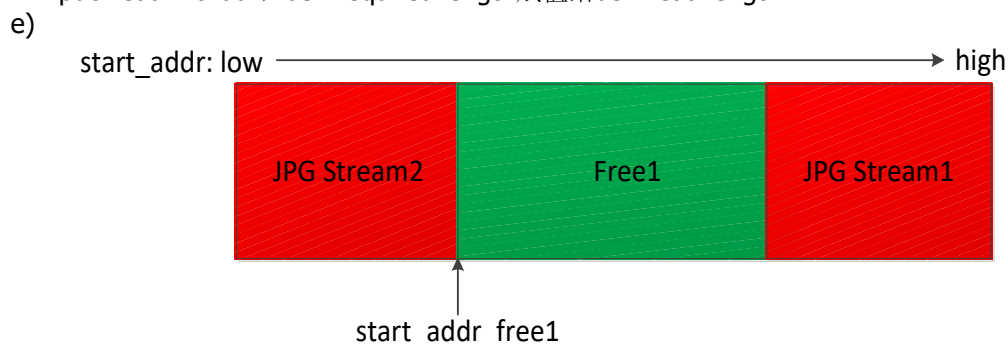
此时底层stream buffer首部存储着一块先前送入但还未解码的JPG码流，如果Free1的长度满足于期望长度u32RequiredLength，底层会将stream buffer的start_addr_free1赋值给pu8HeadVirtAddr，u32RequiredLength赋值给u32HeadLength。



此时底层stream buffer的中间存储着一块先前送入但还未解码的JPG码流，两边都是空闲缓存，如果Free1的长度满足期望长度u32RequiredLength，底层会将stream buffer的start_addr_free1赋值给pu8HeadVirtAddr，u32RequiredLength赋值给u32HeadLength。

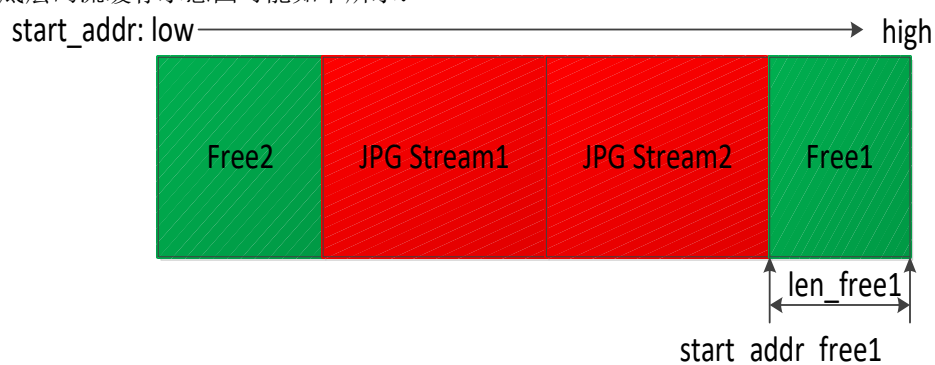


此时底层stream buffer的末尾存储着一块先前送入但还未解码的JPG码流，如果Free1的长度满足期望长度u32RequiredLength，底层会将stream buffer的start_addr赋值给pu8HeadVirtAddr，u32RequiredLength赋值给u32HeadLength。



此时底层stream buffer只有中间一大块空闲缓存，如果Free1的长度满足期望长度u32RequiredLength，底层会将stream buffer的start_addr_free1赋值给pu8HeadVirtAddr，u32RequiredLength赋值给u32HeadLength。

- 如果u32TailLength不为0，说明底层返回两块缓存，两者相加的总长度才满足于期望长度u32RequiredLength，上层需要将一帧码流按序拷贝到pu8HeadVirtAddr和pu8TailVirtAddr。此时底层码流缓存示意图可能如下所示：



中间存储着先前送入但还没解码的JPG码流，头尾两块空闲缓存的总长度加起来才满足期望长度，底层会将 stream buffer 的 start_addr_free1 赋值给 pu8HeadVirtAddr，len_free1 赋值给 u32HeadLength，start_addr赋值给 pu8TailVirtAddr，(u32RequiredLength-u32HeadLength) 赋值给u32TailLength。

- u32ContentLength 表示码流实际有效数据长度，如果 APP 实际传送的码流长度小于 [MI_JPD_GetStreamBuf](#) 中的期望长度 u32RequiredLength，需要将 u32ContentLength 赋值成实际长度，再去调用 [MI_JPD_PutStreamBuf](#)。

➤ 相关数据类型及接口

[MI_JPD_GetStreamBuf](#)
[MI_JPD_PutStreamBuf](#)
[MI_JPD_DropStreamBuf](#)

3.9. MI_JPD_StreamInfo_t

➤ 说明

定义码流信息结构体。

➤ 定义

```
typedef struct MI_JPD_StreamInfo_s
{
    MI_U32 u32CurPicWidth;
    MI_U32 u32CurPicHeight;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U32 u32BufWidth;
    MI_U32 u32BufHeight;
} MI_JPD_StreamInfo_t;
```

➤ 成员

成员名称	描述
u32CurPicWidth	当前解码图像有效宽。
u32CurPicHeight	当前解码图像有效高。
ePixelFormat	图像输入像素存储格式，目前支持semi planar YUV420(即NV12)/YUYV422。参考 MI_SYS_PixelFormat_e 说明。
u32BufWidth	输出缓存实际宽。
u32BufHeight	输出缓存实际高。

※ 注意事项

- 调用 [MI_JPD_DirectBufDecode](#) 时，需要知道输出缓存的大小，该大小是通过 u32BufWidth/u32BufHeight 计算得出，而 u32CurPicWidth/u32CurPicHeight 则是指图像的实际有效宽高。

➤ 相关数据类型及接口

[MI_JPD_ChnAttr_t](#)/[MI_JPD_QueryStreamInfo](#)

3.10. MI_JPD_DirectInputBuf_t

➤ 说明

定义输入码流缓存参数结构体。

➤ 定义

```
typedef struct MI_JPD_DirectInputBuf_s
{
    MI_U8 *pu8InputJPGRawFrameDataVirtAddr;
    MI_PHY phyAddr;
    MI_U32 u32Length;
}MI_JPD_DirectInputBuf_t;
```

➤ 成员

成员名称	描述
pu8InputJPGRawFrameDataVirtAddr	存储JPG码流数据的虚拟地址。
phyAddr	可选，存储JPG码流数据的物理地址，默认为0。
u32Length	码流数据长度。

※ 注意事项

- 该输入码流缓存不能从[MI_JPD_GetStreamBuf](#)获得，而是要通过系统申请。
- 如果一直是这种方式送流解码，[MI_JPD_CreateChn](#)中的参数 u32StreamBufSize 可设置为 0，这样内部就不会分配 stream buffer。
- 通过 [MI_JPD_DirectBufDecode](#) 接口达到同步解码目的。通过 [MI_JPD_GetStreamBuf](#)/[MI_JPD_PutStreamBuf](#) 为异步解码过程。

➤ 相关数据类型及接口

[MI_JPD_QueryStreamInfo](#)/[MI_JPD_DirectBufDecode](#)

3.11. MI_JPD_DirectOutputBuf_t

➤ 说明

定义输出图像缓存参数结构体。

➤ 定义

```
typedef struct MI_JPD_DirectOutputBuf_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U32 u32Width;
    MI_U32 u32Height;
    MI_U32 u32Stride[2];
    MI_PHY phyAddr[2];
}MI_JPD_DirectOutputBuf_t;
```

➤ 成员

成员名称	描述
ePixelFormat	图像输出像素存储格式，目前只支持YUYV422。
u32Width	图像输出缓存实际宽。
u32Height	图像输出缓存实际高。
u32Stride	u32Stride[0]:Y数据stride; u32Stride[1]:UV数据stride。
phyAddr	phyAddr[0]: Y数据的起始物理地址; phyAddr[1]: UV数据的起始物理地址。

※ 注意事项

- 图像输出像素存储格式，目前只支持输出 YUYV422。

➤ 相关数据类型及接口

[MI_JPD_DirectBufDecode](#)

4. 错误码

视频解码错误码如下表所示：

错误代码	宏定义	描述
0xA0202001	MI_ERR_JPD_INVALID_DEVID	设备 ID 超出合法范围
0xA0202002	MI_ERR_JPD_INVALID_CHNID	通道 ID 超出合法范围
0xA0202003	MI_ERR_JPD_ILLEGAL_PARAM	非法参数或输入参数超过通道解码能力
0xA0202004	MI_ERR_JPD_CHN_EXIST	尝试创建已经存在的通道
0xA0202005	MI_ERR_JPD_CHN_UNEXIST	通道不存在
0xA0202006	MI_ERR_JPD_NULL_PTR	输入参数为空指针
0xA0202007	MI_ERR_JPD_NOT_CONFIG	使用前未配置
0xA0202008	MI_ERR_JPD_NOT_SUPPORT	该操作或者功能不支持
0xA0202009	MI_ERR_JPD_NOT_PERM	非法操作
0xA020200C	MI_ERR_JPD_NOMEM	分配内存失败，如系统内存不足
0xA020200D	MI_ERR_JPD_NOBUF	分配缓存失败，如申请的数据缓冲区太大
0xA020200E	MI_ERR_JPD_BUF_EMPTY	缓冲区中无数据
0xA020200F	MI_ERR_JPD_BUF_FULL	缓冲区中数据满
0xA0202010	MI_ERR_JPD_SYS_NOTREADY	系统没有初始化或者相关 依赖的模块没有加载
0xA0202011	MI_ERR_JPD_BADADDR	地址错误
0xA0202012	MI_ERR_JPD_BUSY	系统忙
0xA0202013	MI_ERR_JPD_CHN_NOT_START	通道未使能
0xA0202014	MI_ERR_JPD_CHN_NOT_STOP	通道未禁用
0xA0202015	MI_ERR_JPD_NOT_INIT	模块没有成功加载
0xA0202016	MI_ERR_JPD_INITED	模块已成功加载
0xA020201C	MI_ERR_JPD_CHN_NO_CONTENT	通道无数据
0xA020201F	MI_ERR_JPD_FAILED	失败

5. PROCFS 介绍

5.1. cat

➤ 调试信息

#cat /proc/mi_modules/mi_jpd/mi_jpdN

N 表示不同的 device，不同 chip 的 N 取值如下：

表 5-2 不同 chip 的 N 取值

Chip	N
Tiramisu	0
Muffin	0~3

➤ 调试信息分析

打印分为两部分，使用 Private JpdN Info 分隔开。上半部分为 common 信息，下半部分为 jpd 模块信息。主要记录解码通道的使用情况及配置属性，可用于检查属性配置和当前通道的工作状态，便于 debug。

➤ 参数说明

参数		描述
DEV Info	DevID	硬件设备号，参考表 5-2
	WorkingTaskCnt	该设备正在处理的 task 个数
CHN Info	ChnID	通道号
	Start	使能状态： 0: 不使能 1: 使能
	DecState	解码状态： 0: 解析 header 1: 解码 2: 解码结束
	Width	当前解码图像宽度
	Height	当前解码图像高度
	BufW	当前解码图像需要分配的 output buffer width
	BufH	当前解码图像需要分配的 output buffer height
	StreamBufSize	码流 input buffer pool size，单位：byte
	MinAllocSize	允许成功分配的 input buffer min size
	WorkingTaskCnt	该通道正在处理的 task 个数
	IsrDoneCnt	done 类型的 isr count

参数		描述
	IsrOtherCnt	other 类型的 isr count
	Poll	执行 OnPollingAsyncOutputTaskConfig 的时间点，单位：ms
	Enque	执行 EnqueueAsyncOutputTask 的时间点，单位：ms
	Isr	执行 isr 的时间点，单位：ms
	Check	执行 CheckOutputTaskStatus 的时间点，单位：ms
	Dequeue	执行 DequeueOutputTask 的时间点，单位：ms
	P->E	Poll 到 Enque 的耗时，单位：ms
	E->I	Enque 到 Isr 的耗时，单位：ms
	I->C	Isr 到 Check 的耗时，单位：ms
	C->D	Check 到 Dequeue 的耗时，单位：ms
	P->D	Poll 到 Dequeue 的耗时，单位：ms
	SmoothDecodeTime	平均解码时间，单位：ms
CHN INPUT Info	ChnID	通道号
	SendCnt	App 输入 stream 总个数
	LeftCnt	Input buffer pool 堆积的 stream 个数
	SendStreamSize	App 输入 stream 的总 size
	LeftStreamBytes	Input buffer pool 堆积的 stream size
	EnqueueCnt	执行 EnqueueAsyncOutputTask 成功的总次数
	DirectCnt	调用 MI_JPD_DirectBufDecode 触发解码的次数
	DropCnt	执行 EnqueueAsyncOutputTask 失败的总次数
	RewindCnt	执行 EnqueueAsyncOutputTask rewind 的总次数
	ifps	App 输入 stream 的帧率
	MinIntvl	APP 输入 stream 的最小间隔，单位 ms
	MaxIntvl	APP 输入 stream 的最大间隔，单位 ms
	SumIntvl	APP 输入 stream 的累加间隔，单位 ms
	AvgIntvl	APP 输入 stream 的平均间隔，单位 ms
	Diff	APP 输入 stream 的当前间隔，单位 ms
CHN OUTPUT Info	ChnID	通道号
	DecCnt	解码成功的总次数
	DirectCnt	调用 MI_JPD_DirectBufDecode 解码成功的总次数
	DropCnt	解码失败的总次数
	ofps	输出帧率
	MinIntvl	输出的最小间隔，单位 ms
	MaxIntvl	输出的最大间隔，单位 ms

参数		描述
	SumIntvl	输出的累加间隔, 单位 ms
	AvgIntvl	输出的平均间隔, 单位 ms
	Diff	输出的当前间隔, 单位 ms

5.2. echo

功能	将指定通道的 bitstream buffer 存储到指定路径
命令	echo dumpsb [chn] [bEnable] [path] > /proc/mi_modules/mi_jpd/mi_jpd[dev]
参数说明	[chn]: 通道号 [bEnable]: 1: 开启 dump; 0: 停止 dump [path]: 保存文件的路径 [dev]: 设备号, 参考表 5-2
举例	echo dumpsb 3 1 /mnt > /proc/mi_modules/mi_jpd/mi_jpd0 dump 通道 3 的 bitstream buffer 到/mnt 目录下

功能	将指定通道解码后的输出数据帧存储到指定路径
命令	echo dumpfb [chn] [bEnable] [path] > /proc/mi_modules/mi_jpd/mi_jpd[dev]
参数说明	[chn]: 通道号 [bEnable]: 1: 开启 dump; 0: 停止 dump [path]: 保存文件的路径 [dev]: 设备号, 参考表 5-2
举例	echo dumpfb 3 1 /mnt > /proc/mi_modules/mi_jpd/mi_jpd0 dump 通道 3 的输出数据帧到/mnt 目录下